GUIDE 01

ER Diagram and Monolithic Architecture Design Guide

UniMart Monolithic Full-Stack Lab Series

B.Sc. (Hons) IT - Level 2 | Hands-on Laboratory Practical

Department of Industrial Management
Faculty of Science, University of Kelaniya

Learning outcomes

- Translate the UniMart narrative into a normalized relational data model.
- Create an EER diagram in MySQL Workbench with keys, cardinalities, and constraints.
- Create C4-style context, container, component, and deployment diagrams for a layered monolith.
- Explain how MVC responsibilities map to the Spring Boot monolith.

Prerequisites

- Read the UniMart case study in the supplied software-architecture exercise.
- Basic knowledge of entities, primary keys, foreign keys, and MVC.

Suggested student time: 2.5-3 hours

Scope discipline: The source case defines registration/login, listings, search, messaging, payment/order tracking, reviews, and notifications. The data model below adds only implementation-level fields such as IDs, timestamps, status values, and optimistic-lock versions needed to build those requirements.

1. Convert the case narrative into data concepts

Case statement	Candidate entity or rule
Students register and log in using a university email	User; university_email must be unique; password is stored only as a hash.
Sellers post listings with title, description, price, category and photos	Listing, Category and ListingImage; one seller owns many listings.
Buyers and sellers message each other	Conversation and Message; messages belong to one conversation and one sender.
Buyers pay and orders move through statuses	Order and Payment; payment reference and idempotency key must be unique.
Buyer leaves a review after a transaction	Review belongs to a completed Order; one review per order.
Users receive notifications	Notification belongs to one user and has a type/read state.

2. Recommended entity catalogue

Entity	Minimum attributes	Important constraints
User	id, universityEmail, passwordHash, fullName, role, emailVerified, createdAt, updatedAt	UNIQUE university_email; password_hash never returned by APIs.
Category	id, name, active	UNIQUE name.
Listing	id, sellerId, categoryId, title, description, price, status, createdAt, updatedAt, version	price >= 0; seller/category FKs; version supports optimistic locking.
ListingImage	id, listingId, imageUrl, sortOrder	Delete with listing; URL length limit.
Conversation	id, listingId, buyerId, sellerId, createdAt	Unique buyer/seller/listing combination.
Message	id, conversationId, senderId, messageText, createdAt, readAt	Sender must be a participant.

Entity	Minimum attributes	Important constraints
Order	id, listingId, buyerId, totalAmount, status, paymentMethod, createdAt, updatedAt	Snapshot amount; legal state transitions only.
Payment	id, orderId, amount, status, providerReference, idempotencyKey, paidAt	UNIQUE order_id, provider_reference and idempotency_key.
Review	id, orderId, reviewerId, revieweeId, rating, comment, createdAt, updatedAt	UNIQUE order_id; rating 1-5; completed order required.
Notification	id, userId, type, title, body, isRead, createdAt	Index user_id + is_read + created_at.

3. Reference ER diagram

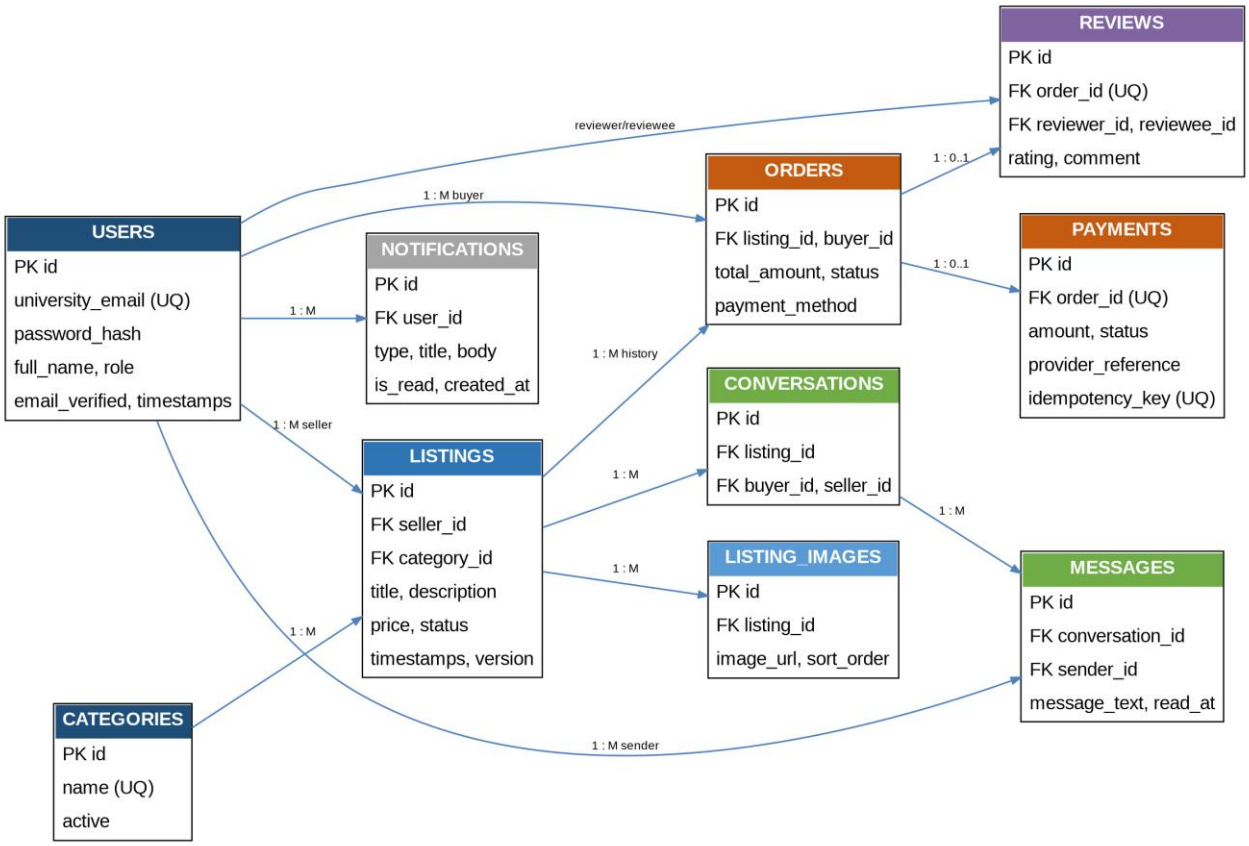


Figure 1. Reference UniMart ER model. Crow-foot cardinalities should be reproduced in MySQL Workbench.

4. Build the EER diagram in MySQL Workbench

1. Open MySQL Workbench and choose File > New Model.
2. Select Add Diagram. Set the schema name to unimart.
3. Create each table from the entity catalogue. Use BIGINT for identifiers, DECIMAL(12,2) for money, VARCHAR for bounded text, TEXT for descriptions/messages, and TIMESTAMP or DATETIME for timestamps.
4. Mark primary keys, NOT NULL fields, unique indexes, and foreign keys before drawing relationships.
5. Use identifying/non-identifying one-to-many relationships carefully. The child has its own surrogate key, so most links are non-identifying.
6. Add indexes for common access paths: listing(status, category_id, created_at), message(conversation_id, created_at), review(reviewee_id, created_at), notification(user_id, is_read, created_at).
7. Use Database > Forward Engineer to review generated DDL. Do not execute until every destructive statement is understood.
8. Export the model as PNG or PDF and save the .mwb model file in the project documentation folder.

Money rule: Do not use FLOAT or DOUBLE for prices. Use DECIMAL so currency values are not changed by binary floating-point rounding.

5. Normalization and integrity checks

- First normal form: each column stores one value; listing photos are rows in listing_images, not comma-separated URLs.
- Second/third normal form: category names are stored once; seller details stay in users; payment details stay in payments.
- Use status enums or constrained strings consistently. Suggested values: Listing AVAILABLE, RESERVED, SOLD, ARCHIVED; Order POSTED, PAID, COMPLETED, CANCELLED; Payment PENDING, SUCCEEDED, FAILED, REFUNDED.
- Prefer soft deletion for listings that have transaction history. Preserve audit evidence rather than physically deleting referenced records.
- Apply business constraints in both the service layer and database where practical.

6. Draw the monolithic architecture

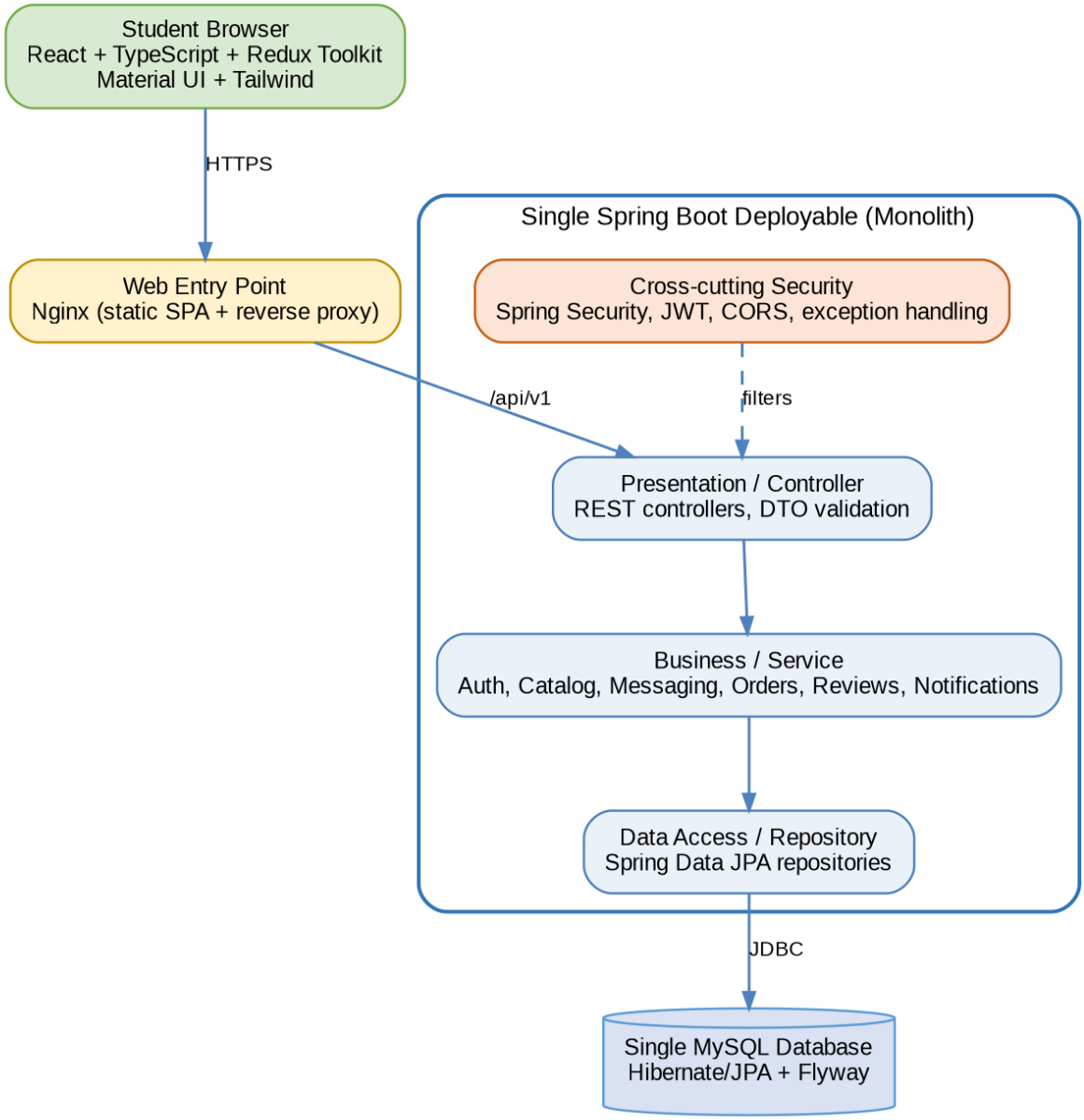


Figure 2. Layered monolith: one Spring Boot deployable and one MySQL database.

Diagram	What to show	Student output
C4 Context	UniMart, students, payment gateway, university email service, notification provider.	One system box and external actors/systems with labelled arrows.
C4 Container	React SPA, Spring Boot monolith, MySQL database, optional object storage/payment gateway.	Technology and protocol labels; only one backend deployable.
Component / package view	Auth, catalog, messaging, orders/payments, reviews, notifications inside the monolith.	Show Controller -> Service -> Repository dependencies.
Deployment	Browser, Nginx, one Spring Boot process/container, one MySQL instance on one VM.	Ports, public/private boundaries, and environment variables.

7. MVC mapping used in this lab

MVC responsibility	Spring Boot implementation	Rule
Controller	@RestController classes and request/response DTOs	Handles HTTP only; no database calls and minimal business logic.
Model	JPA entities, DTOs, domain rules and repositories	Entities are persistence models; do not serialize entities directly.
Service	@Service classes with @Transactional methods	Owns use-case logic, authorization decisions and orchestration.
View	React SPA	Backend returns JSON; React renders the user interface.

8. Diagram quality checklist

- Every relationship shown in the diagram exists in the written model and vice versa.
- All arrows are labelled with a protocol or purpose; avoid unexplained lines.
- The monolith is visibly one deployable even though it contains multiple modules.
- The database is one shared schema for the monolithic stage.
- Security boundaries are visible: browser/public network, reverse proxy, backend, private database.
- Names are consistent across ER diagram, code packages, API paths, and UI features.

Completion checklist

☐ EER diagram contains PKs, FKs, nullability, unique indexes and cardinalities.

☐ Context and container diagrams are legible at A4 size.

☐ Component diagram maps to the planned package structure.

☐ Deployment diagram shows one backend deployable and one database.

☐ A one-paragraph rationale explains why monolith-first fits a small student team.